

How we record our meetings

A working local meeting recorder that has captured three real sessions. What it does, how it does it, what it produced, and where it falls down.

WHAT I'M ASKING

This has been running on my Mac for about six weeks and it works well enough that I use it for real calls. It is not a product — it is five shell scripts and a local model. I want your read on whether it is worth turning into one.

If yes, there is real work to do and I've scoped it. If no, that is a perfectly good answer — I'll keep using it as-is, because it already pays for itself in my own meetings.

01 — THE IDEA

Everything stays on the machine

The tool records a meeting, transcribes it, watches the screen, and writes structured notes. All of it runs locally on the laptop — the audio never leaves the machine, there is no account, no server, and no per-minute cost. It is not a bot that joins your call; nothing appears in the participant list, because it is just recording the computer.

The parts are all open-source and already installed: **ffmpeg 7.1.1** for capture, **whisper.cpp 1.9.1** compiled native arm64 with Metal for speech-to-text, **tesseract 5.5.2** for reading the screen, and **Ollama 0.32.4** running a local language model for the notes. Nothing here is a paid API.

The reason I built it this way rather than using Otter or Read AI: I was on calls I did not want to send to a third party, and I wanted the links and figures that get shared on screen — which no transcription tool captures, because they are never spoken aloud.

02 — MECHANISM

Four things run in parallel during a call

When a meeting starts, four loops kick off against the same session. They are independent — if one dies, the others keep going.

AUDIO	ffmpeg records the microphone continuously to a growing WAV file. ffmpeg -f avfoundation -i :0 → 16 kHz mono
TRANSCRIPT	Every 150 seconds, a loop measures how much new audio has appeared, slices out only that new window, and sends it to a whisper server running in the background. The result is appended to a running transcript. It only transcribes what it hasn't seen — so cost stays flat no matter how long the meeting runs. 150 s cycle · 25 s minimum window · whisper-server on localhost
SCREEN	Every 25 seconds it takes a screenshot, runs OCR on it, extracts any URLs it finds, and keeps unique on-screen lines. The screenshot is deleted immediately after reading. This is the part that catches links pasted in chat, figures on slides, and product names nobody says out loud. 25 s cycle · tesseract · PNG discarded after OCR
VIDEO	Optional. Records the screen in segments that auto-restart if one stops, then stitches them together at the end. Off by default — it produces large files and is only worth it when you need to re-read something. segmented screencapture · off unless asked for

When the meeting ends, a final pass reads the whole transcript plus everything OCR'd off the screen, and writes the notes: summary, what was discussed, decisions, questions and answers, a table of figures, links, and action items.

THE ONE DESIGN DECISION WORTH KNOWING ABOUT

The notes mark **where each number came from** — `[OCR]` if it was read off the screen, `[audio]` if it was heard. Spoken numbers get mistranscribed constantly ("36 triple 9" for ₹36,999). Screen-read numbers are exact. Marking the difference means you know which figures you can trust without going back to the recording. No commercial tool I've looked at does this.

03 — OUTPUT

What you get afterwards

Every session leaves a folder. These are the actual file sizes from the 9 August capture, a 75-minute webinar:

ARTIFACTS FROM ONE 75-MINUTE SESSION		
FILE	SIZE	WHAT IT IS
<code>notes.md</code>	7.2 KB	The written notes. This is the thing you actually read
<code>transcript.md</code>	25 KB	Raw timestamped transcript — the ground truth to check against
<code>screen-text.txt</code>	108 KB	Everything OCR'd off the screen during the call
<code>links.md</code>	1.4 KB	Deduplicated URLs with first-seen timestamps
<code>.wav</code>	1.3 GB	The recording. Delete or archive after — this is the storage cost

The notes are markdown, so they render anywhere and go straight into a doc or a PDF. The audio is the only heavy thing; everything readable is under 150 KB per meeting.

04 — EVIDENCE

Three real sessions

Rather than describe what it can do, here is what it actually did. Including the one that failed, which taught us the most.

15 Jul · 2h 15m **Online workshop**

FAILED

Captured almost no speech. AirPods were the output device, so everyone else's audio went into the headphones and never reached the microphone — the recorder was only ever hearing my side of a call where I barely spoke. Two and a quarter hours produced about 1.2 KB of sound-effect tags.

Then it got worse. The summariser took that near-empty transcript and invented a complete, confident, entirely fictional set of notes — tools, links, step-by-step instructions, none of it real. Nothing in the output indicated it was fabricated. Only the OCR'd links were genuine.

6 Aug · 34m **Vendor sales call**

PARTIAL

Clean transcript for the first 17 minutes, then the other side's audio genuinely failed and the rest was unusable. Not our fault — the call broke — but it showed something useful: **OCR carried the session after audio died.** Their portal URLs, the platform they were using and their community link all came off the screen, not the microphone.

The notes from this one were good enough that I used them to decide not to proceed with the vendor.

Full 75 minutes of clean audio, no dropout. The notes came out genuinely useful — the offer structure, the pricing ladder, the claims made, the persuasion tactics, and a set of action items. Every financial figure was cross-checked against what was on screen.

The limitation showed here too: the presenter switched between Hindi and English constantly, and roughly a fifth of the speech was lost or mangled because the model we were running was **English-only**.

05 — FIXES

What those three sessions changed

I fixed four things this week off the back of them. All committed and pushed.

- **It now refuses to write notes from silence.** Before summarising, it strips sound-effect tags and counts actual speech. Below a threshold it writes a diagnostic instead of notes — telling you to check your capture devices. Tested against all three sessions: the July one is refused at 361 characters of real speech; the other two pass at 7,067 and 24,888. The hallucination cannot repeat.
- **Hindi.** The model was English-only, and on top of that the language was hard-coded to English — two separate bugs, either of which alone would have blocked Hindi. Now running a multilingual model with auto-detection. On the same 30-second Hindi-English clip: **33 characters transcribed before, 481 after.**
- **The notes template was hard-coded to one specific event** — it produced workshop-shaped notes for every recording regardless of what it was. That is why the sales-call notes had to be written by hand. It now adapts to the kind of session.
- **A stop bug** left a background transcription server running for 23 days across sessions, holding the wrong model.

Two servers now run instead of one: a small fast English model for everyday voice, and the large multilingual one loaded only while a meeting is actually

recording — it needs 1.6 GB and there is no reason to hold that all day.

06 — HONEST LIMITS

Where it falls down

Taking these in order of how much they'd matter to anyone other than me:

- **It only hears my side properly.** The AirPods failure is not fully fixed. Capturing the computer's own audio output on a Mac needs a specific system API, and that work is designed but not built. Right now, if your headphones are the output device, the other side may not be recorded. This is the single biggest gap.
- **It doesn't know who is speaking.** One microphone, one merged stream, no speaker labels. So no "Kishore committed to X" — just that it was said. Fixable, but it is real work and will never be as accurate as tools that get per-speaker audio from Zoom directly.
- **Mac only, and a capable Mac.** The multilingual model needs about 1.6 GB of memory and real silicon. On an ordinary Windows laptop it either runs very slowly or has to fall back to a model that cannot do Hindi at all.
- **It is five shell scripts.** No interface, no installer. You start it from a terminal. Fine for me, not fine for anyone else.
- **Hindi is better, not solved.** The multilingual model is a large improvement but standard speech models still handle Hindi-English switching poorly. Getting this genuinely good likely needs an India-specific model.
- **Nobody has been told they're being recorded.** For my own use that is fine — recording a call you're part of is legal in India. As something other people run against third parties, disclosure needs to be built in, not assumed. Two competitors are in court over exactly this.

07 — COST

What it costs to run

Nothing, which is the interesting part. No subscription, no API, no per-minute charge. A 75-minute meeting costs electricity and about 1.3 GB of disk until you delete the audio.

For comparison, Read AI — the closest commercial equivalent — is **\$15 to \$39.75 per person per month**, caps free accounts at five meetings, and puts video playback behind its \$29.75 tier.

BUT THE COST ARGUMENT IS WEAKER THAN IT LOOKS

I should be straight about this because it changes the case for building. Hosted transcription has become very cheap — around **\$0.04 per hour of audio**. Twenty hours of meetings a month costs about 80 cents to transcribe in the cloud. So "we're free and they're not" is not much of an advantage; commercial tools are not priced on transcription cost. The genuine advantages are that **nothing leaves the machine**, that it **reads the screen**, and that it **works when meeting platforms block recording bots** — which Teams and Google Meet both started doing this year.

Two reasonable answers

Option A

Keep it as an internal tool

Cost: nothing. It already works for the person using it. I'd finish the both-sides audio capture so it stops missing half of calls, and leave the rest alone.

Perfectly defensible. It has already paid for itself — the notes from the 6 August call directly informed a decision not to spend money.

Option B

Turn it into something we can give people

Roughly six to ten weeks: rewrite the scripts as a proper cross-platform command-line tool, add Windows support, add speaker labels, build the report properly, and design disclosure in from the start.

- Windows is *easier* than Mac for this, oddly — capturing system audio there needs no permission at all
- Machines that can't run it locally would fall back to hosted transcription at about \$0.80 per user per month

My own read: **Option A is not a failure state.** The thing works and I will keep using it either way. Option B is worth doing only if we think there is a customer — and that assumption is completely untested. Nobody outside this building has ever asked for it.

What would help

- **Try it on one of your calls.** That is the fastest way to judge it — I can set it up on your machine if it's a Mac, and you'll have notes within a few minutes of hanging up. Everything above is easier to assess with a real output in front of you.

- **Tell me whether the notes are actually good.** I've read all three and I am not a neutral judge. If they read as noise to you, that settles it.
- **If you lean towards Option B** — the honest first question is not technical. It's who pays for this, and I don't have an answer yet.

There is a deeper write-up on how this compares to the commercial tools, and a separate one on what a cross-platform version would take. Happy to send either if useful, but this is the part that matters first.

All figures measured on the actual system — an M1 Mac, 16 GB, whisper.cpp 1.9.1 native arm64 with Metal — against the three real captures described above, not from documentation or vendor claims. Code is on `feat/meetings-m1`. Recordings and transcripts stay on my machine and are excluded from the repository.

Soumya Swain · soumya@suryaai.co.in